

Neural Network Implementation using TensorFlow

1. Objective

The objective of this task is to understand the fundamentals of neural networks and deep learning by implementing a basic neural network model using TensorFlow. The task involves learning the architecture of neural networks, activation functions, backpropagation, and training a model to classify handwritten digits from the MNIST dataset.

2. Introduction

Deep Learning is a subset of Artificial Intelligence (AI) that enables computers to learn from large amounts of data using artificial neural networks. Neural networks are inspired by the structure of the human brain and consist of interconnected neurons that process information.

In this project, a basic neural network was developed using TensorFlow and Keras to classify handwritten digits from the MNIST dataset. The model learns patterns in the training images and predicts the correct digit for unseen images with high accuracy.

3. Dataset Description

The MNIST dataset is one of the most widely used datasets for learning image classification. It contains grayscale images of handwritten digits ranging from 0 to 9.

Dataset Details

- Dataset Name: MNIST Handwritten Digits
- Total Images: 70,000
- Training Images: 60,000
- Testing Images: 10,000
- Image Size: 28×28 pixels
- Number of Classes: 10 (Digits 0–9)

4. Libraries Used

The following Python libraries were used during the implementation:

- NumPy
- Matplotlib
- TensorFlow
- Keras
- Scikit-learn
- Seaborn

Each library plays an important role in data processing, visualization, model development, and evaluation.

5. Data Preprocessing

Before training the neural network, the dataset was preprocessed using the following steps:

- Loaded the MNIST dataset from TensorFlow.
- Checked the shape of the training and testing datasets.
- Displayed sample handwritten digit images.
- Normalized pixel values from the range of 0–255 to 0–1 to improve model performance.

6. Neural Network Architecture

The neural network consists of the following layers:

Layer	Description
Flatten Layer	Converts the 28×28 image into a one-dimensional vector of 784 values.
Dense Layer (128 Neurons)	First hidden layer using the ReLU activation function.
Dense Layer (64 Neurons)	Second hidden layer using the ReLU activation function.
Output Layer (10 Neurons)	Uses the Softmax activation function to classify digits into one of the 10 classes.

7. Components of a Neural Network

Artificial Neuron

An artificial neuron is the fundamental unit of a neural network. It receives input values, performs mathematical computations using weights and biases, applies an activation function, and generates an output.

Input Layer

The input layer receives the input data. In this project, each handwritten image of size 28×28 pixels is provided as input to the neural network.

Hidden Layers

Hidden layers process the input data and learn meaningful patterns. The model contains two hidden layers with 128 and 64 neurons, respectively.

Output Layer

The output layer contains 10 neurons representing the digits from 0 to 9. The neuron with the highest probability is selected as the predicted class.

Activation Function

Activation functions introduce non-linearity into the neural network, enabling it to learn complex relationships.

ReLU (Rectified Linear Unit)

- Used in hidden layers.
- Converts negative values to zero while keeping positive values unchanged.
- Helps the network learn efficiently.

Softmax

- Used in the output layer.
- Converts outputs into probability values.
- The class with the highest probability is selected as the final prediction.

Loss Function

The loss function measures the difference between the predicted output and the actual output. This project uses **Sparse Categorical Cross Entropy**, which is suitable for multi-class classification problems.

Optimizer

The optimizer updates the model's weights during training to minimize the loss. The **Adam optimizer** is used because it provides fast and efficient convergence.

Backpropagation

Backpropagation is the learning process in which the neural network calculates errors and updates its weights using gradient descent to improve prediction accuracy.

Epoch

An epoch represents one complete pass of the training dataset through the neural network.

Accuracy

Accuracy is the percentage of correctly classified samples out of the total number of samples.

8. Model Training

The neural network was compiled using the Adam optimizer and Sparse Categorical Cross entropy loss function. The model was trained for **10 epochs**, with **20% of the training data reserved for validation**. During training, both training accuracy and validation accuracy were monitored to evaluate the learning performance of the model.

9. Results

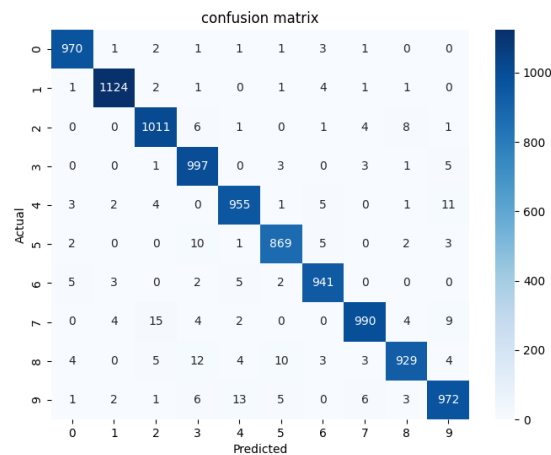
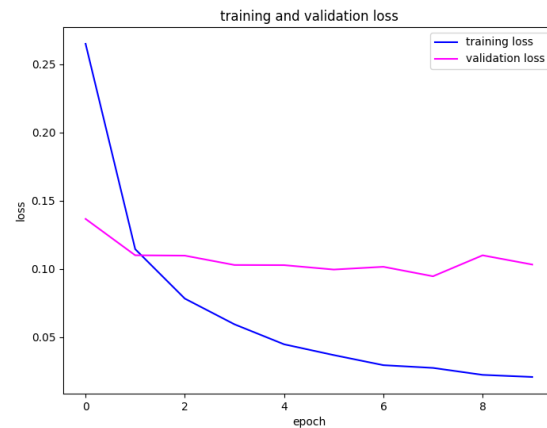
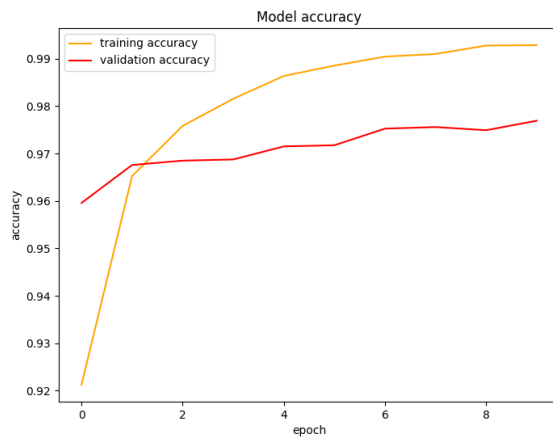
The trained neural network achieved a high classification accuracy on the MNIST test dataset.

Test Accuracy: 97.57%

The following visualizations were generated during the implementation:

- Training Accuracy Graph
- Training Loss Graph
- Confusion Matrix

- Classification Report



10. Conclusion

In this task, a basic neural network was successfully implemented using TensorFlow to classify handwritten digits from the MNIST dataset. The model demonstrated excellent performance by learning patterns from the training data and accurately predicting unseen images.

This project provided practical experience in building neural networks, understanding activation functions, model training, backpropagation, and evaluating deep learning models. It also strengthened the understanding of the core concepts of deep learning and image classification.